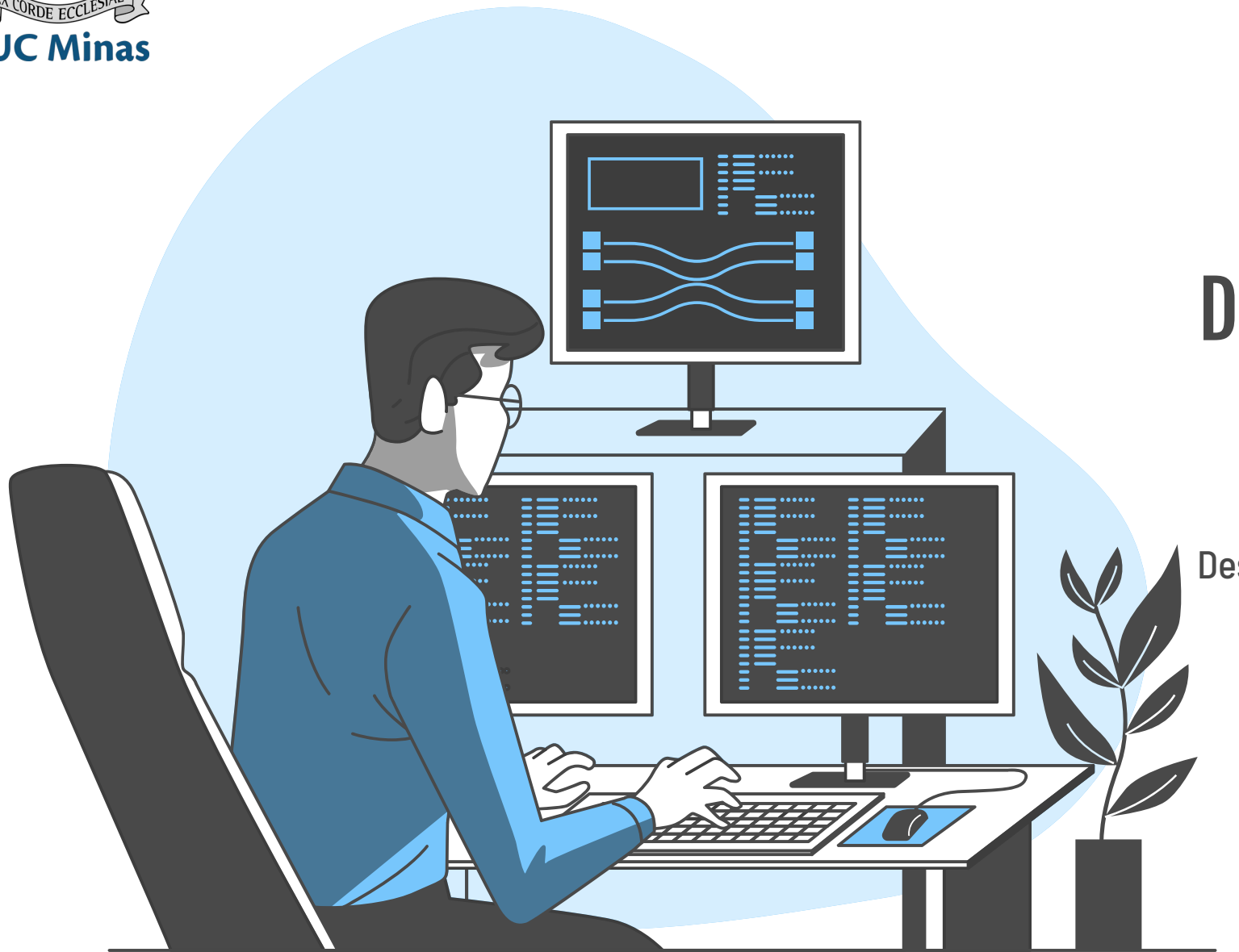
PUC Minas



Aula 10 · FullStack

Documentando a API com OpenAPI e Swagger UI

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender o que é a especificação **OpenAPI**

2

Adicionar o **springdoc-openapi** ao Spring Boot

3

Usar o **Swagger UI** para explorar e testar a API

4

Descrever endpoints e modelos com `@Operation`, `@ApiResponse` e `@Schema`

5

Configurar o botão **Authorize** para rotas com JWT

Antes de começar

- Back-end, aulas 05 e 10
- Aula 01 desta pasta (contrato da API)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O contrato precisa estar escrito

02 Passo 1: dependência

03 Passo 2: abrir o Swagger UI

04 Passo 3: testar pelo navegador

05 Passo 4: descrevendo melhor os endpoints

06 Passo 5: informações gerais e JWT

07 Configurações úteis

08 Aprofundando: o contrato a serviço do front

01

O contrato precisa estar escrito

O contrato precisa estar escrito

Na aula 01 escrevemos o contrato da API à mão no README. Isso funciona, mas envelhece: alguém muda um endpoint e esquece de atualizar a tabela. A **OpenAPI** é um formato padrão (JSON/YAML) para descrever uma API REST: rotas, parâmetros, corpos, respostas e segurança. Com o **springdoc**, esse documento é **gerado a partir do próprio código**, então nunca fica desatualizado.

01

O contrato precisa estar escrito

Aula 10 · FullStack

O contrato precisa estar escrito

Termo	O que é
OpenAPI	A especificação (o formato do documento), antiga “Swagger Specification”
springdoc-openapi	Biblioteca que lê seus controllers e gera o documento OpenAPI
Swagger UI	Página web que mostra o documento e permite testar cada endpoint
`/v3/api-docs`	O documento em JSON
`/swagger-ui.html`	A interface visual

02

Passo 1: dependência

Passo 1: dependência

pom.xml

```xml
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.0.0</version>   <!-- linha 3.x: Spring Boot 4 | linha 2.x: Spring Boot 3 -->
</dependency>
```

Passo 1: dependência

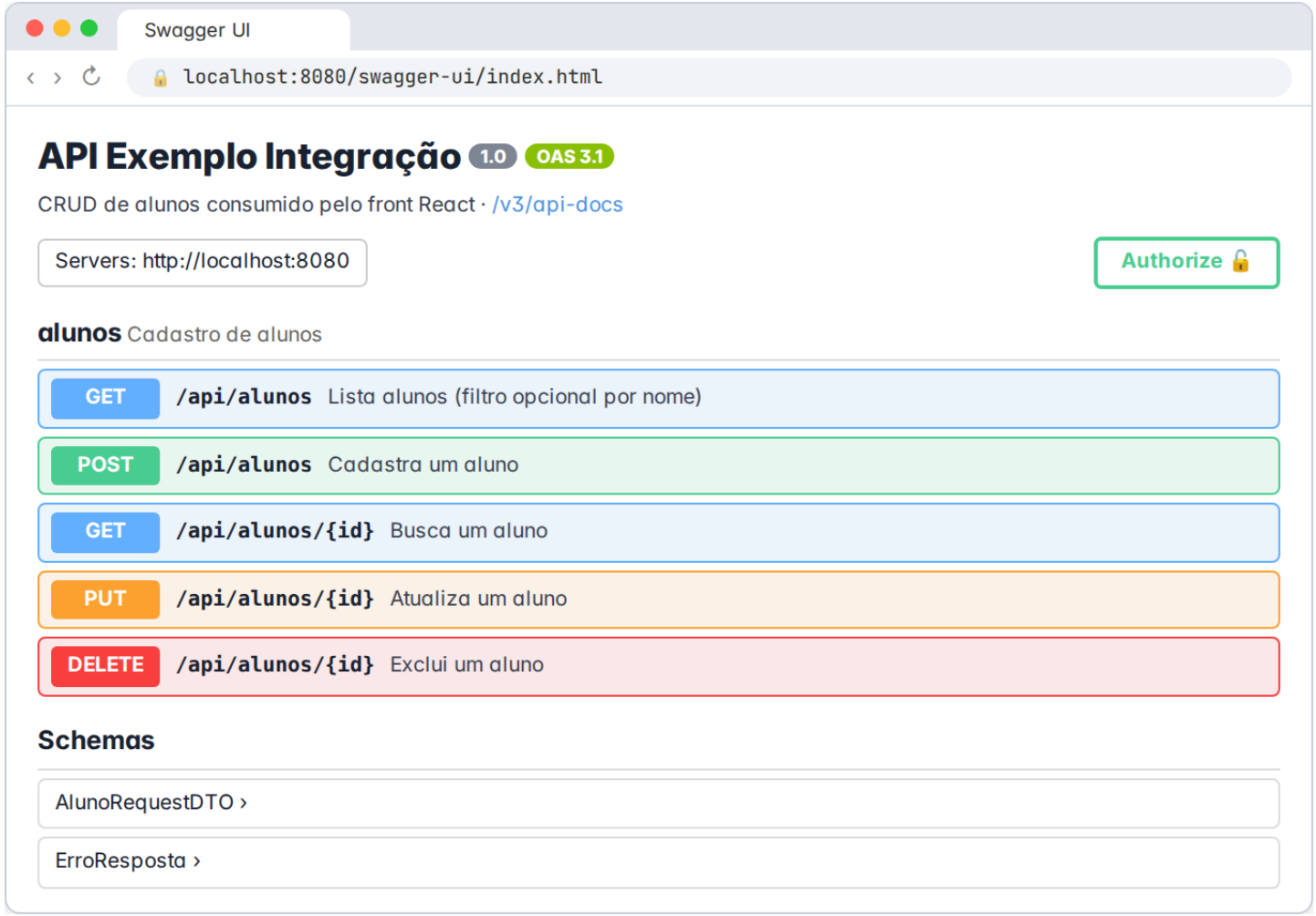
Atenção

A versão do springdoc acompanha a do Spring Boot: **2.x para Boot 3** (como no projeto `Documentation_JWT_RestAPI`, que usa 2.3.0) e **3.x para Boot 4**. Usar a linha errada costuma dar `ClassNotFoundException` na inicialização. Confira a versão mais recente em *springdoc.org*.

03

Passo 2: abrir o Swagger UI

Passo 2: abrir o Swagger UI



Sem escrever nada além da dependência, suba o projeto e acesse `http://localhost:8080/swagger-ui.html`. Todos os controllers aparecem agrupados, com os verbos coloridos.

Swagger UI gerado a partir dos controllers. As cores seguem os verbos HTTP.

04

Passo 3: testar pelo navegador

Passo 3: testar pelo navegador

1. Clique em um endpoint (ex.: **POST /api/alunos**) para expandir.
2. Clique em **Try it out**: os campos ficam editáveis.
3. Edite o JSON de exemplo do corpo e clique em **Execute**.
4. Veja logo abaixo o comando `curl` equivalente, a URL chamada, o **status** e o corpo da resposta.
5. Teste também os erros (corpo vazio, id inexistente) e confira se o JSON de erro segue o contrato.

05

Passo 4: descrevendo melhor os endpoints

Passo 4: descrevendo melhor os endpoints

controller/AlunoController.java

```java
@Tag(name = "alunos", description = "Cadastro de alunos")
@RestController
@RequestMapping("/api/alunos")
public class AlunoController {

    @Operation(summary = "Cadastra um aluno",
               description = "Nome e e-mail são obrigatórios; o e-mail precisa ser válido.")
    @ApiResponse(responseCode = "201", description = "Aluno criado")
    @ApiResponse(responseCode = "400", description = "Dados inválidos",
                 content = @Content(schema = @Schema(implementation = ErroResposta.class)))
    @PostMapping
    public ResponseEntity<Aluno> inserir(@Valid @RequestBody AlunoRequestDTO dto) { ... }

    @Operation(summary = "Lista alunos (filtro opcional por nome)")
    @GetMapping
    public List<Aluno> listar(
        @Parameter(description = "Trecho do nome, sem diferenciar maiúsculas", example = "ana")
        @RequestParam(required = false) String nome) { ... }
}
```

Passo 4: descrevendo melhor os endpoints

dto/AlunoRequestDTO.java

```java
@Schema(description = "Dados para cadastrar ou atualizar um aluno")
public record AlunoRequestDTO(
    @Schema(example = "Ana Souza") @NotBlank String nome,
    @Schema(example = "ana@email.com") @NotBlank @Email String email,
    @Schema(example = "Engenharia de Software", nullable = true) String curso) { }
```

Passo 4: descrevendo melhor os endpoints

Dica

O springdoc lê as anotações de validação (`@NotBlank`, `@Email`, `@Size`) e já marca os campos como obrigatórios e com formato na documentação. Bons DTOs geram boa documentação de graça.

06

Passo 5: informações gerais e JWT

Passo 5: informações gerais e JWT

config/OpenApiConfig.java

```java
@Configuration
@OpenAPIDefinition(info = @Info(
        title = "API Exemplo Integração", version = "1.0",
        description = "CRUD de alunos consumido pelo front React",
        contact = @Contact(name = "Prof. Dr. João Paulo Aramuni")))
@SecurityScheme(name = "bearerAuth", type = SecuritySchemeType.HTTP,
                scheme = "bearer", bearerFormat = "JWT")
public class OpenApiConfig { }

// nos controllers protegidos:
@SecurityRequirement(name = "bearerAuth")
@RestController
@RequestMapping("/api/admin")
public class AdminController { ... }
```

Passo 5: informações gerais e JWT

1. Faça login em **POST /api/login** pelo próprio Swagger e copie o `token` da resposta.
2. Clique no botão **Authorize** (cadeado) no topo da página.
3. Cole o token no campo **bearerAuth** (sem a palavra `Bearer`) e confirme.
4. Todas as rotas com `@SecurityRequirement` passam a enviar `Authorization: Bearer ...` automaticamente.

Passo 5: informações gerais e JWT

SecurityConfig: libere a documentação

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers(HttpMethod.POST, "/api/login").permitAll()
    .anyRequest().authenticated());
```

07

Configurações úteis

Configurações úteis

application.properties

```
# caminho da interface (padrão /swagger-ui.html)
springdoc.swagger-ui.path=/docs
# ordena endpoints por método e tags por nome
springdoc.swagger-ui.operations-sorter=method
springdoc.swagger-ui.tags-sorter=alpha
# documenta só a API (não o actuator)
springdoc.paths-to-match=/api/**
# desliga em produção, se a API não for pública
springdoc.api-docs.enabled=${DOCS_ENABLED:true}
springdoc.swagger-ui.enabled=${DOCS_ENABLED:true}
```

Configurações úteis

Erro comum

`/swagger-ui.html` **dá 401**: o Spring Security está barrando; libere os caminhos acima. **Página em branco ou 404**: versão do springdoc incompatível com o Boot (veja o aviso do Passo 1). **Endpoints não aparecem**: a classe principal fica em outro pacote e o `scanBasePackages` não inclui os controllers, ou `paths-to-match` está filtrando demais.

08

Aprofundando: o contrato a serviço do front

Aprofundando: o contrato a serviço do front

Trecho de /v3/api-docs

```
{
  "openapi": "3.1.0",
  "info": { "title": "API Exemplo Integração", "version": "1.0" },
  "paths": {
    "/api/alunos": {
      "post": {
        "tags": ["alunos"],
        "summary": "Cadastra um aluno",
        "requestBody": { "content": { "application/json": {
          "schema": { "$ref": "#/components/schemas/AlunoRequestDTO" } } } },
        "responses": { "201": { "description": "Aluno criado" },
                       "400": { "description": "Dados inválidos" } }
      }
    }
  }
}
```

O que fazer com o documento OpenAPI

Importar no Insomnia/Postman

Import → *URL* `http://localhost:8080/v3/api-docs`: todas as requisições prontas.

Gerar cliente do front

`openapi-typescript` gera os tipos; o front sabe os campos de cada resposta.

Revisar mudanças

Salvar o JSON no repositório mostra no *diff* quando o contrato mudou.

Mocks

Ferramentas como Prism sobem uma API falsa a partir do documento, para o front trabalhar antes do back ficar pronto.

Aprofundando: o contrato a serviço do front

Tipos do front gerados a partir da API

```
npx openapi-typescript http://localhost:8080/v3/api-docs -o src/api/schema.d.ts

// no front (com TypeScript), os tipos vêm do back:
import type { components } from "./api/schema";
type Aluno = components["schemas"]["AlunoResponseDTO"];
```

Documentação manual x gerada

Manual (README)

- Livre para explicar contexto e decisões
- Envelhece quando o código muda
- Ótima para visão geral e como rodar

vs

Gerada (springdoc)

- Sempre igual ao código
- Permite testar pelo navegador
- Ótima para o detalhe de cada endpoint

Aprofundando: o contrato a serviço do front

Anotação	Onde	Para quê
`@Tag`	Controller	Agrupa endpoints
`@Operation`	Método	Resumo e descrição
`@Parameter`	Parâmetro	Descrição e exemplo
`@ApiResponse`	Método	Cada status possível
`@Schema`	DTO e campos	Descrição, exemplo, obrigatoriedade
`@Hidden`	Controller ou método	Esconde da documentação

Aprofundando: o contrato a serviço do front

Na prática

O projeto `SpringBoot/Documentation_JWT_RestAPI` combina tudo desta aula: login JWT, Swagger UI com o botão Authorize e JavaDoc. Compare com o `JavaDoc_JWT_RestAPI` para ver a diferença entre documentar **o código** (JavaDoc, para quem mantém) e documentar **a API** (OpenAPI, para quem consome).

09

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

O que é OpenAPI?

A Um banco de dados

B Um formato padrão para descrever APIs REST

C Uma biblioteca React

D Um provedor de nuvem

Quiz · questão 1 de 5

Resposta

O que é OpenAPI?

A Um banco de dados

✓ Um formato padrão para descrever APIs REST

C Uma biblioteca React

D Um provedor de nuvem

Por quê: O Swagger UI exibe esse documento.

Pergunta

Qual a linha do springdoc para Spring Boot 4?

A 1.x

B 2.x

C 3.x

D Qualquer uma

Resposta

Qual a linha do springdoc para Spring Boot 4?

A 1.x

B 2.x

✓ 3.x

D Qualquer uma

Por quê: A 2.x é para Boot 3.

Pergunta

Em qual endereço fica a interface padrão?

A /api/docs

B /swagger-ui.html

C /v3

D /actuator

Resposta

Em qual endereço fica a interface padrão?

A /api/docs

✓ /swagger-ui.html

C /v3

D /actuator

Por quê: O JSON fica em /v3/api-docs.

Pergunta

Por que o Swagger UI responde 401?

A springdoc quebrado

B O Spring Security não liberou os caminhos da documentação

C Falta CORS

D Banco offline

Resposta

Por que o Swagger UI responde 401?

A springdoc quebrado

✓ O Spring Security não liberou os caminhos da documentação

C Falta CORS

D Banco offline

Por quê: Libere /swagger-ui/ **e /v3/api-docs/**.

Pergunta

Para que servem `@SecurityScheme` e `@SecurityRequirement`?

A Criar usuários

B Habilitar o botão Authorize e enviar o Bearer nas rotas protegidas

C Criptografar senhas

D Gerar o token

Resposta

Para que servem `@SecurityScheme` e `@SecurityRequirement`?

A Criar usuários

✓ Habilitar o botão Authorize e enviar o Bearer nas rotas protegidas

C Criptografar senhas

D Gerar o token

Por quê: Cole o token obtido no /api/login.

Fechamento

Resumo da aula

- **OpenAPI** é o formato padrão para descrever APIs; o **springdoc** gera o documento a partir dos controllers.
- Dependência `springdoc-openapi-starter-webmvc-ui` (3.x no Boot 4); interface em `/swagger-ui.html`, JSON em `/v3/api-docs`.
- **Try it out** testa endpoints pelo navegador e mostra o curl equivalente.
- `@Tag`, `@Operation`, `@ApiResponse` e `@Schema` enriquecem a documentação; validações aparecem sozinhas.
- `@SecurityScheme` + `@SecurityRequirement` habilitam o botão **Authorize** para JWT; libere as rotas da documentação no Security.

Exercícios

1. Adicione o springdoc ao seu back e envie o print do Swagger UI com todos os endpoints.

2. Documente o POST e o PUT com `@Operation`, respostas 201/200/400/404 e exemplos nos DTOs.

3. Configure o JWT no Swagger e teste uma rota protegida pelo botão Authorize.

4. Importe o `/v3/api-docs` no Insomnia e compare com a coleção que você tinha feito à mão.

Para ir além

- springdoc.org
- swagger.io/specification
- Projeto `SpringBoot/Documentation_JWT_RestAPI`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 10?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web